Process & Decision Documentation

Week 6 Side Quests

Focus on:

- Build an interactive game that controlled by arrow keys.
- Generate a level using arrays, use loops to dynamically place tiles.
- Bonus Integration: Add physics-based smooth movement (lerp) and particle collision effects to make the game reactive.

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: Mar 9, 2026

Tool Disclosure: Gemini Pro

Purpose of Use: Class content code explanation, generating and translating code comments from Chinese to English, refining documentation text, and implementing the physics bonus requirement (smooth movement and particle collision effects).

Summary of Interaction: The AI explained the logic and provided a step-by-step map out the direction of what I should do for this side quest. I also instructed the AI to write clear code comments to explain the logic and then translate all my Chinese comments into English for the final submission. Also, the AI provided the mathematical logic to add lerp-based smooth movement and a particle system that reacts to wall collisions and level completion.

Human Decision Point(s): I rejected the advice from AI to add console logs to clean up the code. I requested the addition of on-screen instructions to improve user experience. For the bonus physics update, I choosing to focus solely on visual physics. I also decided to keep my original level design rather than adding more obstacles to maintain the current game balance.

Integrity & Verification Note: I verified the code by running it in VS Code. I tested the detection by moving the object into walls and confirmed that once the object reaches the yellow square, the game correctly triggers the level switch as intended. I also verified that the Chinese-to-English translated comments accurately reflected the code's function and tested that the new particle effects trigger correctly upon hitting walls and the goal.

Scope of GenAI Use: The AI assisted with modifying the logic as I required, including generating the particle system, lerp logic, and writing/translating code comments. The project setup, decision-making, and final testing were performed manually by me.

Limitations or Misfires: The AI initially explained a complex code structure and suggested adding audio components. I directed it to simplify by using plain language to match my current understanding of the course material and removed the sound logic entirely to keep the project focused.

Appendix

You said

hi read through the side quest instruction. Ive gave you my previous sketch please based on my sketch and assist me to modify :

Add sound or simple physics to your sketch

e.g., visual ehects triggered by motion, collisions, or player actions. Bonus: Combine both: use sound and physics for a reactive, multi-sensory ehect.

Gemini said

This is a super fun side quest! Since you already did a fantastic job separating your data and logic using JSON, adding these extra layers will really bring your game to life.

To help you get that **Bonus point**, we are going to combine both **physics** (smooth movement and particle explosions) and **sound** (retro synth noises triggered by your actions).

Here is the step-by-step plan to upgrade your sketch.

Step 1: Add the p5.sound Library to `index.html`

To use sound without needing external MP3 files, we can use the `p5.sound` library to synthesize retro game noises on the fly! You just need to add one line to your `<head>` tag.

Update your `index.html` to look like this:

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Week 4 Side Quest</title>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.4.0/p5.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.4.0/addons/p5.sound.min.j
s"></script>
    <link rel="stylesheet" type="text/css" href="style.css">
  </head>
  <body>
    <script src="sketch.js"></script>
  </body>
</html>
```

Step 2: Update `sketch.js` (The Engine)

We are going to add three major features here:

1. **Physics (Lerping):** Instead of instantly teleporting from square to square, the player will slide smoothly.
2. **Visual Effects (Particles):** When you bump into a wall or hit the goal, a burst of particles will spawn.
3. **Sound (Oscillators):** We will generate short beep/boop sounds for moving, bumping, and winning.

Replace your entire `sketch.js` with this upgraded version:

JavaScript

```
let levelData;
let currentLevel = 0;
let tileSize = 50;
// Added vr and vc for "visual row" and "visual col" to handle smooth physics
sliding
let player = { r: 1, c: 1, vr: 1, vc: 1 };

// --- NEW: Physics & Sound Variables ---
let particles = [];
let osc; // Oscillator for retro sound effects

function preload() {
  levelData = loadJSON('levels.json');
```

```
}

function setup() {
  createCanvas(400, 300);
  noStroke();

  // Set up the synthesizer for sound effects
  osc = new p5.Oscillator('square');
  osc.amp(0); // Start silent
  osc.start();

  resetPlayer();
}

function draw() {
  background(220);

  let grid = levelData.levels[currentLevel].grid;

  // 1. Draw the map
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      let type = grid[r][c];
      let x = c * tileSize;
      let y = r * tileSize;

      if (type === 1) fill(50);
      else if (type === 2) fill(100, 255, 100);
      else if (type === 3) fill(255, 215, 0);
      else fill(255);

      rect(x, y, tileSize, tileSize);
    }
  }

  // --- NEW: Physics (Smooth Movement using Lerp) ---
  // This smoothly transitions the visual position to the actual grid
position
  player.vr = lerp(player.vr, player.r, 0.3);
  player.vc = lerp(player.vc, player.c, 0.3);

  // Draw player using visual coordinates
  fill(0);
  rect(player.vc * tileSize + 10, player.vr * tileSize + 10, tileSize - 20,
tileSize - 20);

  // --- NEW: Visual Effects (Particles) ---
  // Loop backward to draw and manage particle lifespans
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.x += p.vx; // Move by velocity (physics)
    p.y += p.vy;
    p.alpha -= 10; // Fade out

    fill(p.color[0], p.color[1], p.color[2], p.alpha);
    rect(p.x, p.y, p.size, p.size);
```

```
    // Remove dead particles
    if (p.alpha <= 0) particles.splice(i, 1);
  }

  // Instructions
  fill(50);
  textSize(16);
  textAlign(CENTER, CENTER);
  text("Use Arrow Keys to Move. Reach Gold to Win!", width / 2, height - 25);
}

function keyPressed() {
  // Browsers require user interaction to play audio, this activates it
  userStartAudio();

  let grid = levelData.levels[currentLevel].grid;
  let nextR = player.r;
  let nextC = player.c;

  if (keyCode === LEFT_ARROW)  nextC--;
  if (keyCode === RIGHT_ARROW) nextC++;
  if (keyCode === UP_ARROW)    nextR--;
  if (keyCode === DOWN_ARROW)  nextR++;

  // Safety check: ensure we are within grid bounds
  if (nextR >= 0 && nextR < grid.length && nextC >= 0 && nextC <
grid[0].length) {
    if (grid[nextR][nextC] !== 1) {
      player.r = nextR;
      player.c = nextC;

      // Check for goal
      if (grid[nextR][nextC] === 3) {
        playSound('win');
        spawnParticles(player.c * tileSize + 25, player.r * tileSize + 25,
[255, 215, 0]); // Gold explosion
        setTimeout(nextLevel, 400); // Slight delay so you can see the
explosion!
      } else {
        playSound('move');
      }
    } else {
      // Hit a wall!
      playSound('bump');
      // Spawn grey wall particles at the boundary
      let hitX = ((player.c + nextC) / 2) * tileSize + 25;
      let hitY = ((player.r + nextR) / 2) * tileSize + 25;
      spawnParticles(hitX, hitY, [100, 100, 100]);
    }
  }
}

// --- NEW: Helper function to spawn physics particles ---
function spawnParticles(x, y, colorArr) {
  for (let i = 0; i < 15; i++) {
    particles.push({
      x: x,
```

```
      y: y,
      vx: random(-3, 3), // Random velocity X
      vy: random(-3, 3), // Random velocity Y
      alpha: 255,
      size: random(4, 8),
      color: colorArr
    });
  }
}

// --- NEW: Helper function to trigger synthesized sound effects ---
function playSound(type) {
  if (type === 'move') {
    osc.freq(400); // High pitch
    osc.amp(0.1, 0.01); // Fade in fast
    osc.amp(0, 0.1, 0.1); // Fade out
  } else if (type === 'bump') {
    osc.freq(150); // Low pitch
    osc.amp(0.2, 0.01);
    osc.amp(0, 0.1, 0.1);
  } else if (type === 'win') {
    osc.freq(600); // Happy high pitch
    osc.amp(0.2, 0.01);
    osc.freq(800, 0.1); // Slide the pitch up!
    osc.amp(0, 0.3, 0.2);
  }
}

function nextLevel() {
  currentLevel++;
  if (currentLevel >= levelData.levels.length) {
    currentLevel = 0;
  }
  resetPlayer();
}

function resetPlayer() {
  let grid = levelData.levels[currentLevel].grid;
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      if (grid[r][c] === 2) {
        player.r = r;
        player.c = c;
        // Reset visual positions so we don't slide from the previous level's
exit
        player.vr = r;
        player.vc = c;
        return;
      }
    }
  }
}
```

What We Changed:

- **The `p5.sound` Oscillator:** You now have a built-in synth generating low thuds for walls, short blips for walking, and a happy slide-up sound for winning.
- **The Particle System:** When you try to walk into a wall, it calculates the boundary and shoots out grey particles with basic velocity physics. Winning shoots out gold particles.
- **Lerp Physics:** `lerp()` smoothly animates a value between two points over time. We split your player's actual position (`r`,`c`) from their visual position (`vr`,`vc`) so the math handles the smooth sliding animation automatically.

Would you like me to draft an update for your `README.md` and your `Process_and_Decision_Documentation` to reflect these new features for your submission?

You said

Do not add soud effect for me, I want to focus on my physics effect upgrade.

Gemini said

No problem! Removing the sound will definitely keep the code cleaner and avoid introducing unnecessary libraries. We can focus on physics and visual effects. In that case, your index.html doesn't need any changes; just keep the way you uploaded it.

JavaScript

```
let levelData;
let currentLevel = 0;
let tileSize = 50;
// 增加了 vr (visual row) 和 vc (visual col) 来处理平滑的物理滑动效果
let player = { r: 1, c: 1, vr: 1, vc: 1 };

// --- 新增: 物理/视觉效果变量 ---
let particles = []; // 用于存储粒子效果的数组

function preload() {
  levelData = loadJSON('levels.json');
}

function setup() {
  createCanvas(400, 300);
  noStroke();
  resetPlayer();
}
```

```
function draw() {
  background(220);

  let grid = levelData.levels[currentLevel].grid;

  // 1. 画地图
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      let type = grid[r][c];
      let x = c * tileSize;
      let y = r * tileSize;

      if (type === 1) fill(50);
      else if (type === 2) fill(100, 255, 100);
      else if (type === 3) fill(255, 215, 0);
      else fill(255);

      rect(x, y, tileSize, tileSize);
    }
  }

  // --- 新增：物理（使用 Lerp 实现平滑移动）---
  // 这会让玩家的视觉位置(vr, vc)平滑地过渡到实际的网格位置(r, c)
  player.vr = lerp(player.vr, player.r, 0.3);
  player.vc = lerp(player.vc, player.c, 0.3);

  // 使用视觉坐标（vc, vr）来画玩家，感受滑行的物理反馈
  fill(0);
  rect(player.vc * tileSize + 10, player.vr * tileSize + 10, tileSize - 20,
tileSize - 20);

  // --- 新增：视觉效果（粒子系统）---
  // 倒序循环，方便在粒子消失时将它们从数组中移除
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.x += p.vx; // 根据速度移动（物理 X 轴）
    p.y += p.vy; // 根据速度移动（物理 Y 轴）
    p.alpha -= 10; // 逐渐变透明

    fill(p.color[0], p.color[1], p.color[2], p.alpha);
    rect(p.x, p.y, p.size, p.size);

    // 移除已经完全透明（消失）的粒子
    if (p.alpha <= 0) particles.splice(i, 1);
  }

  // 底部说明文字
  fill(50);
  textSize(16);
  textAlign(CENTER, CENTER);
```

```
  text("Use Arrow Keys to Move. Reach Gold to Win!", width / 2, height - 25);
}

function keyPressed() {
  let grid = levelData.levels[currentLevel].grid;
  let nextR = player.r;
  let nextC = player.c;

  if (keyCode === LEFT_ARROW)  nextC--;
  if (keyCode === RIGHT_ARROW) nextC++;
  if (keyCode === UP_ARROW)    nextR--;
  if (keyCode === DOWN_ARROW)  nextR++;

  // 安全检查：确保没有越界
  if (nextR >= 0 && nextR < grid.length && nextC >= 0 && nextC <
grid[0].length) {
    if (grid[nextR][nextC] !== 1) {
      player.r = nextR;
      player.c = nextC;

      // 检查是否到达终点
      if (grid[nextR][nextC] === 3) {
        // 触发金色粒子爆炸
        spawnParticles(player.c * tileSize + 25, player.r * tileSize + 25,
[255, 215, 0]);
        setTimeout(nextLevel, 400); // 稍微延迟一下过关，这样你能看清爆炸效果！
      }
    } else {
      // 撞墙啦！
      // 在玩家和墙壁的交界处生成灰色的墙壁粉末/碎屑粒子
      let hitX = ((player.c + nextC) / 2) * tileSize + 25;
      let hitY = ((player.r + nextR) / 2) * tileSize + 25;
      spawnParticles(hitX, hitY, [100, 100, 100]);
    }
  }
}

// --- 新增：用于生成物理粒子效果的辅助函数 ---
function spawnParticles(x, y, colorArr) {
  for (let i = 0; i < 15; i++) {
    particles.push({
      x: x,
      y: y,
      vx: random(-3, 3), // X 轴随机速度
      vy: random(-3, 3), // Y 轴随机速度
      alpha: 255,        // 初始不透明度
      size: random(4, 8),// 随机大小
      color: colorArr
    });
  }
}
```

```
function nextLevel() {
  currentLevel++;
  if (currentLevel >= levelData.levels.length) {
    currentLevel = 0;
  }
  resetPlayer();
}

function resetPlayer() {
  let grid = levelData.levels[currentLevel].grid;
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      if (grid[r][c] === 2) {
        player.r = r;
        player.c = c;
        // 重置视觉坐标，防止从上一关的出口滑行到这一关的入口
        player.vr = r;
        player.vc = c;
        return;
      }
    }
  }
}
```

You said

把 comment 翻译成英文：

```
let levelData;
let currentLevel = 0;
let tileSize = 50;
// 增加了 vr (visual row) 和 vc (visual col) 来处理平滑的物理滑动效果
let player = { r: 1, c: 1, vr: 1, vc: 1 };

// --- 新增：物理/视觉效果变量 ---
let particles = []; // 用于存储粒子效果的数组

function preload() {
levelData = loadJSON('levels.json');
}

function setup() {
createCanvas(400, 300);
noStroke();
resetPlayer();
}

function draw() {
background(220);

let grid = levelData.levels[currentLevel].grid;
```

```
// 1. 画地图
for (let r = 0; r < grid.length; r++) {
for (let c = 0; c < grid[r].length; c++) {
let type = grid[r][c];
let x = c * tileSize;
let y = r * tileSize;

if (type === 1) fill(50);
else if (type === 2) fill(100, 255, 100);
else if (type === 3) fill(255, 215, 0);
else fill(255);

rect(x, y, tileSize, tileSize);
}
}

// --- 新增: 物理 (使用 Lerp 实现平滑移动) ---
// 这会让玩家的视觉位置(vr, vc)平滑地过渡到实际的网格位置(r, c)
player.vr = lerp(player.vr, player.r, 0.3);
player.vc = lerp(player.vc, player.c, 0.3);

// 使用视觉坐标 (vc, vr) 来画玩家, 感受滑行的物理反馈
fill(0);
rect(player.vc * tileSize + 10, player.vr * tileSize + 10, tileSize - 20,
tileSize - 20);

// --- 新增: 视觉效果 (粒子系统) ---
// 倒序循环, 方便在粒子消失时将它们从数组中移除
for (let i = particles.length - 1; i >= 0; i--) {
let p = particles[i];
p.x += p.vx; // 根据速度移动 (物理 X 轴)
p.y += p.vy; // 根据速度移动 (物理 Y 轴)
p.alpha -= 10; // 逐渐变透明

fill(p.color[0], p.color[1], p.color[2], p.alpha);
rect(p.x, p.y, p.size, p.size);

// 移除已经完全透明 (消失) 的粒子
if (p.alpha <= 0) particles.splice(i, 1);
}

// 底部说明文字
fill(50);
textSize(16);
textAlign(CENTER, CENTER);
text("Use Arrow Keys to Move. Reach Gold to Win!", width / 2, height - 25);
}

function keyPressed() {
let grid = levelData.levels[currentLevel].grid;
let nextR = player.r;
```

```
let nextC = player.c;

if (keyCode === LEFT_ARROW) nextC--;
if (keyCode === RIGHT_ARROW) nextC++;
if (keyCode === UP_ARROW) nextR--;
if (keyCode === DOWN_ARROW) nextR++;

// 安全检查：确保没有越界
if (nextR >= 0 && nextR < grid.length && nextC >= 0 && nextC <
grid[0].length) {
if (grid[nextR][nextC] !== 1) {
player.r = nextR;
player.c = nextC;

// 检查是否到达终点
if (grid[nextR][nextC] === 3) {
// 触发金色粒子爆炸
spawnParticles(player.c * tileSize + 25, player.r * tileSize + 25, [255, 215,
0]);
setTimeout(nextLevel, 400); // 稍微延迟一下过关，这样你能看清爆炸效果！
}
} else {
// 撞墙啦！
// 在玩家和墙壁的交界处生成灰色的墙壁粉末/碎屑粒子
let hitX = ((player.c + nextC) / 2) * tileSize + 25;
let hitY = ((player.r + nextR) / 2) * tileSize + 25;
spawnParticles(hitX, hitY, [100, 100, 100]);
}
}
}

// --- 新增：用于生成物理粒子效果的辅助函数 ---
function spawnParticles(x, y, colorArr) {
for (let i = 0; i < 15; i++) {
particles.push({
x: x,
y: y,
vx: random(-3, 3), // X 轴随机速度
vy: random(-3, 3), // Y 轴随机速度
alpha: 255, // 初始不透明度
size: random(4, 8),// 随机大小
color: colorArr
});
}
}

function nextLevel() {
currentLevel++;
if (currentLevel >= levelData.levels.length) {
currentLevel = 0;
}
```

```
resetPlayer();
}

function resetPlayer() {
let grid = levelData.levels[currentLevel].grid;
for (let r = 0; r < grid.length; r++) {
for (let c = 0; c < grid[r].length; c++) {
if (grid[r][c] === 2) {
player.r = r;
player.c = c;
// 重置视觉坐标，防止从上一关的出口滑行到这一关的入口
player.vr = r;
player.vc = c;
return;
}
}
}
}
```

Gemini said

Here is your sketch.js code with all the Chinese comments translated into English:

JavaScript

```
let levelData;
let currentLevel = 0;
let tileSize = 50;
// Added vr (visual row) and vc (visual col) to handle smooth physics sliding effects
let player = { r: 1, c: 1, vr: 1, vc: 1 };

// --- NEW: Physics/Visual effects variables ---
let particles = []; // Array to store particle effects

function preload() {
  levelData = loadJSON('levels.json');
}

function setup() {
  createCanvas(400, 300);
  noStroke();
  resetPlayer();
}

function draw() {
  background(220);

  let grid = levelData.levels[currentLevel].grid;

  // 1. Draw the map
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      let type = grid[r][c];
      let x = c * tileSize;
      let y = r * tileSize;

      if (type === 1) fill(50);
      else if (type === 2) fill(100, 255, 100);
```

```
      else if (type === 3) fill(255, 215, 0);
      else fill(255);

      rect(x, y, tileSize, tileSize);
    }
  }

  // --- NEW: Physics (Smooth Movement using Lerp) ---
  // This smoothly transitions the visual position (vr, vc) to the actual
grid position (r, c)
  player.vr = lerp(player.vr, player.r, 0.3);
  player.vc = lerp(player.vc, player.c, 0.3);

  // Draw player using visual coordinates (vc, vr) to show the sliding
physical feedback
  fill(0);
  rect(player.vc * tileSize + 10, player.vr * tileSize + 10, tileSize - 20,
tileSize - 20);

  // --- NEW: Visual Effects (Particle System) ---
  // Reverse loop, making it easier to remove particles from the array when
they disappear
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.x += p.vx; // Move based on velocity (Physics X-axis)
    p.y += p.vy; // Move based on velocity (Physics Y-axis)
    p.alpha -= 10; // Fade out gradually

    fill(p.color[0], p.color[1], p.color[2], p.alpha);
    rect(p.x, p.y, p.size, p.size);

    // Remove particles that are completely transparent (disappeared)
    if (p.alpha <= 0) particles.splice(i, 1);
  }

  // Instructions text at the bottom
  fill(50);
  textSize(16);
  textAlign(CENTER, CENTER);
  text("Use Arrow Keys to Move. Reach Gold to Win!", width / 2, height - 25);
}

function keyPressed() {
  let grid = levelData.levels[currentLevel].grid;
  let nextR = player.r;
  let nextC = player.c;

  if (keyCode === LEFT_ARROW)  nextC--;
  if (keyCode === RIGHT_ARROW) nextC++;
  if (keyCode === UP_ARROW)    nextR--;
  if (keyCode === DOWN_ARROW)  nextR++;

  // Safety check: ensure we are within grid bounds
  if (nextR >= 0 && nextR < grid.length && nextC >= 0 && nextC <
grid[0].length) {
    if (grid[nextR][nextC] !== 1) {
      player.r = nextR;
```

```
      player.c = nextC;

      // Check if the goal is reached
      if (grid[nextR][nextC] === 3) {
        // Trigger gold particle explosion
        spawnParticles(player.c * tileSize + 25, player.r * tileSize + 25,
[255, 215, 0]);
        setTimeout(nextLevel, 400); // Slight delay for the level transition
so you can see the explosion effect!
      }
    } else {
      // Hit a wall!
      // Generate grey wall powder/debris particles at the boundary between
the player and the wall
      let hitX = ((player.c + nextC) / 2) * tileSize + 25;
      let hitY = ((player.r + nextR) / 2) * tileSize + 25;
      spawnParticles(hitX, hitY, [100, 100, 100]);
    }
  }
}

// --- NEW: Helper function used to generate physical particle effects ---
function spawnParticles(x, y, colorArr) {
  for (let i = 0; i < 15; i++) {
    particles.push({
      x: x,
      y: y,
      vx: random(-3, 3), // Random X-axis velocity
      vy: random(-3, 3), // Random Y-axis velocity
      alpha: 255,        // Initial opacity
      size: random(4, 8),// Random size
      color: colorArr
    });
  }
}

function nextLevel() {
  currentLevel++;
  if (currentLevel >= levelData.levels.length) {
    currentLevel = 0;
  }
  resetPlayer();
}

function resetPlayer() {
  let grid = levelData.levels[currentLevel].grid;
  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[r].length; c++) {
      if (grid[r][c] === 2) {
        player.r = r;
        player.c = c;
        // Reset visual coordinates to prevent sliding from the previous
level's exit to the current level's entrance
        player.vr = r;
        player.vc = c;
        return;
      }
```

```
    }
  }
}
```

Would you like me to review any other parts of your code before you finalize this side quest?